

Sistemas de Computación

Práctico # 5 Device drivers

Grupo: NoTengoGrupo

Alumnos:

- Bernardi Mateo
- Regnicoli Gustavo
- Schreiner Federico

Docentes:

- Miguel Angel Solinas
- Javier Alejandro Jorge

Año: 2026

Introducción:

Un "driver" es aquel que conduce, administra, controla, dirige, monitorea la entidad bajo su mando. Un "bus driver" hace eso con un "bus". De manera similar, un "device driver" hace eso con un dispositivo. Un dispositivo puede ser cualquier periférico conectado a una computadora, por ejemplo, un mouse, un teclado, una pantalla/monitor, un disco duro, una cámara, un reloj, etc., cualquier cosa.

Un "driver" puede ser una persona o sistemas automáticos, posiblemente monitoreados por otra persona. Del mismo modo, el "device driver" podría ser una pieza de software u otro periférico/dispositivo, posiblemente controlado por un software. Sin embargo, si se trata de otro periférico/dispositivo, se denomina "device controller" en el lenguaje común. Y por "driver" solo nos referimos a un "software driver". Un "device controller" es un dispositivo en sí mismo y, por lo tanto, muchas veces también necesita un "driver", comúnmente conocido como "bus driver".

Los ejemplos generales de "device controller" incluyen controladores de disco duro, controladores de pantalla, controladores de audio para los dispositivos correspondientes. Ejemplos más técnicos serían los controladores para los protocolos de hardware, como un controlador IDE, un controlador PCI, un controlador USB, un controlador SPI, un controlador I2C, etc.

En el desarrollo de esta clase veremos estas sutiles diferencias y aprenderemos a construir un "driver" de caracteres.

Para superar este TP tendrán que diseñar y construir un CDD que permita sensar dos señales externas con un periodo de UN segundo. Luego una aplicación a nivel de usuario deberá leer UNA de las dos señales y graficarla en función del tiempo. La aplicación también debe poder indicarle al CDD cuál de las dos señales leer. Las correcciones de escalas de las mediciones, de ser necesario, se harán a nivel de usuario. Los gráficos de la señal deben indicar el tipo de señal que se está sensando, unidades en abscisas y tiempo en ordenadas. Cuando se cambie de señal el gráfico se debe "resetear" y acomodar a la nueva medición.

Desarrollo:

Este trabajo práctico consiste en diseñar y construir un Character Device Driver (CDD) para Linux que permita sensar dos señales externas con un período de un segundo. Una aplicación de usuario lee una de las dos señales seleccionadas y la grafica en tiempo real vía interfaz web.

En primer lugar se descarga la imagen oficial de Raspberry Pi OS y los archivos de kernel necesarios para QEMU:

```
cd ~/noTengoGrupo-SdC/tp5_device_drivers

mkdir qemu-raspi && cd qemu-raspi

# Se descarga imagen oficial (Raspberry Pi OS Lite, 32-bit, es la más
compatible con QEMU)
wget
https://downloads.raspberrypi.com/raspios_lite_armhf/images/raspios_lite_ar
mhf-2024-11-19/2024-11-19-raspios-bookworm-armhf-lite.img.xz
```

```
# Se descomprime
xz -d 2024-11-19-raspbios-bookworm-armhf-lite.img.xz

# Renombrar para comodidad
mv 2024-11-19-raspbios-bookworm-armhf-lite.img raspi.img
```

QEMU necesita el kernel (kernel) y el device tree blob (.dtb) por separado. Los bajamos del repositorio oficial de la Raspi

```
wget
https://github.com/dhruvvyas90/qemu-rpi-kernel/raw/master/kernel-qemu-4.19.50-buster
wget
https://github.com/dhruvvyas90/qemu-rpi-kernel/raw/master/versatile-pb-buster.dtb

# Renombrar para claridad
mv kernel-qemu-4.19.50-buster kernel-qemu
mv versatile-pb-buster.dtb raspi.dtb

# Agrandar imagen 2GB más
qemu-img resize raspi.img +2G

# Verificar tamaño
qemu-img info raspi.img
```

```
gustavo@gustavo-Lenovo-V145-15AST:~/noTengoGrupo-SdC/tp5_device_drivers/qemu-raspi$ qemu-img info raspi.img
image: raspi.img
file format: raw
virtual size: 4.38 GiB (4697620480 bytes)
disk size: 1.56 GiB
Child node '/file':
  filename: raspi.img
  protocol type: file
  file length: 4.38 GiB (4697620480 bytes)
  disk size: 1.56 GiB
```

```
gustavo@gustavo-Lenovo-V145-15AST:~/noTengoGrupo-SdC/tp5_device_drivers/qemu-raspi$ fdisk -l raspi.img
Disco raspi.img: 4,38 GiB, 4697620480 bytes, 9175040 sectores
Unidades: sectores de 1 * 512 = 512 bytes
Tamaño de sector (lógico/físico): 512 bytes / 512 bytes
Tamaño de E/S (mínimo/óptimo): 512 bytes / 512 bytes
Tipo de etiqueta de disco: dos
Identificador del disco: 0x57b902f5

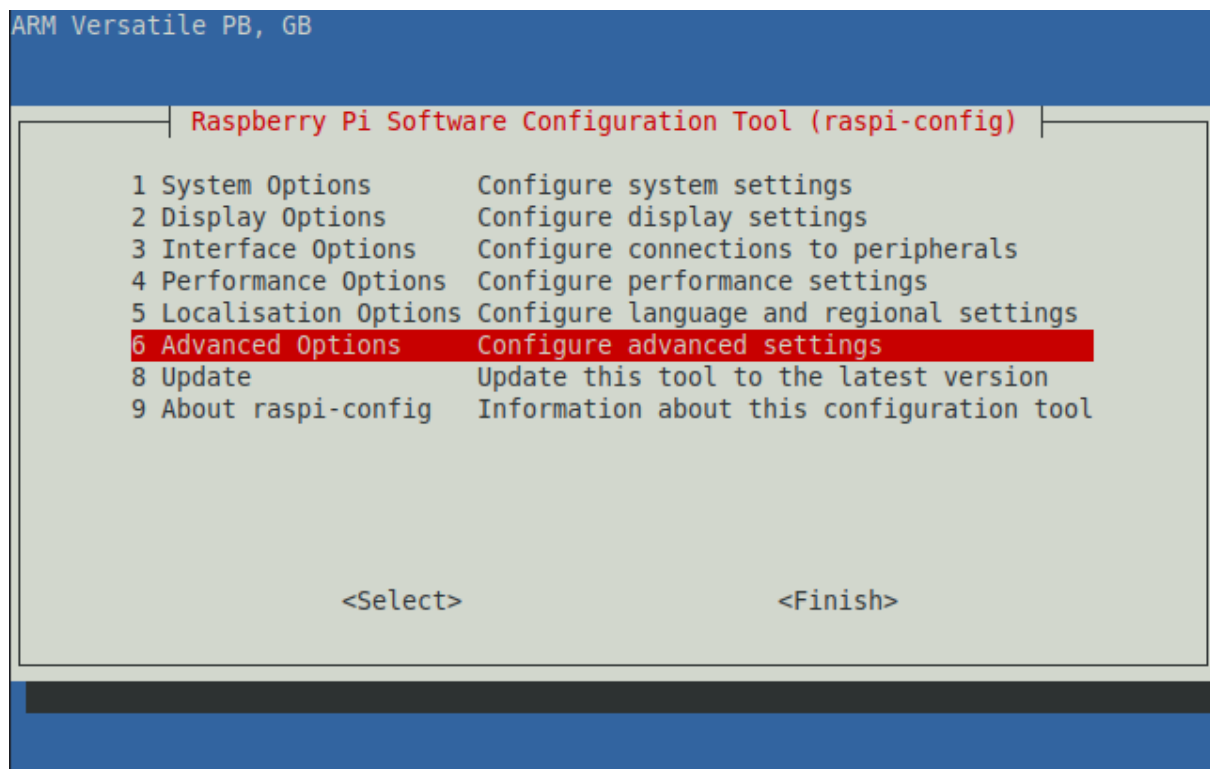
Disposit.  Inicio  Comienzo  Final  Sectores  Tamaño  Id  Tipo
raspi.img1      8192 1056767 1048576    512M    c  W95 FAT32 (LBA)
raspi.img2    1056768 4980735 3923968    1,9G   83  Linux
```

Se monta la partición boot de la imagen y se crea el archivo ssh para que el servicio SSH arranque:

```
sudo mkdir -p /mnt/raspi-boot
sudo mount -o loop,offset=4194304 raspi.img /mnt/raspi-boot
sudo touch /mnt/raspi-boot/ssh
sudo umount /mnt/raspi-boot
```

El kernel nativo se extrae directamente de la imagen para garantizar compatibilidad exacta con el sistema instalado:

```
sudo mount -o loop,offset=4194304 raspi.img /mnt/raspi-boot
cp /mnt/raspi-boot/kernel.img ./kernel-raspi-native
sudo umount /mnt/raspi-boot
```



El script de arranque de QEMU redirige los puertos necesarios para SSH y la interfaz web: start-raspi.sh

```
qemu-system-arm \
-m 256 \
-machine versatilepb \
-cpu arm1176 \
-kernel kernel-raspi-native \
-dtb raspi.dtb \
-drive file=raspi.img,format=raw \
-append "root=/dev/sda2 panic=1 rootfstype=ext4 rw" \
-net nic \
-net user,hostfwd=tcp:127.0.0.1:5022-:22,hostfwd=tcp:127.0.0.1:8080-:8080 \
-no-reboot
```



```
systemd[1]: Starting fake-hwclock.service - Restore / save the current clock...
systemd[1]: Starting keyboard-setup.service - Set the console keyboard layout...
systemd[1]: kmmod-static-nodes.service - Create List of Static Device Nodes was skipped because of an unmet condition check (ConditionFileNotEmpty=/lib/modules/4.19.50+/modules.devname).
systemd[1]: Starting modprobe@configfs.service - Load Kernel Module configfs...
systemd[1]: Starting modprobe@dm_mod.service - Load Kernel Module dm_mod...
systemd[1]: Starting modprobe@drm.service - Load Kernel Module drm...
systemd[1]: Starting modprobe@efi_pstore.service - Load Kernel Module efi_pstore...
systemd[1]: Starting modprobe@fuse.service - Load Kernel Module fuse...
systemd[1]: Starting modprobe@loop.service - Load Kernel Module loop...
systemd[1]: systemd-fsck-root.service - File System Check on Root Device was skipped because of an unmet condition check (ConditionPathsReadWrite=1/).
systemd[1]: systemd-journald.service: unit configures an IP firewall, but the local system does not support BPF/cgroup firewalling.
systemd[1]: (This warning is only shown for the first unit using IP firewalling.)
systemd[1]: Starting systemd-journald.service - Journal Service...
systemd[1]: Starting systemd-modules-load.service - Load Kernel Modules...
systemd[1]: Starting systemd-remount-fs.service - Remount Root and Kernel File Systems...
systemd[1]: Starting systemd-udev-trigger.service - Coldplug All udev Devices...
systemd[1]: Mounted dev-mqueue.mount - POSIX Message Queue File System.
systemd[1]: Mounted sys-kernel-debug.mount - Kernel Debug File System.
systemd[1]: Finished fake-hwclock.service - Restore / save the current clock.
systemd[1]: modprobe@configfs.service: Deactivated successfully.
systemd[1]: Finished modprobe@configfs.service - Load Kernel Module configfs.
systemd[1]: modprobe@dm_mod.service: Deactivated successfully.
systemd[1]: Finished modprobe@dm_mod.service - Load Kernel Module dm_mod.
systemd[1]: modprobe@drm.service: Deactivated successfully.
systemd[1]: Finished modprobe@drm.service - Load Kernel Module drm.
systemd[1]: modprobe@efi_pstore.service: Deactivated successfully.
systemd[1]: Finished modprobe@efi_pstore.service - Load Kernel Module efi_pstore.
systemd[1]: modprobe@fuse.service: Deactivated successfully.
systemd[1]: Finished modprobe@fuse.service - Load Kernel Module fuse.
systemd[1]: modprobe@loop.service: Deactivated successfully.
systemd[1]: Finished modprobe@loop.service - Load Kernel Module loop.
systemd[1]: Finished systemd-modules-load.service - Load Kernel Modules.
systemd[1]: sys-fs-fuse-connections.mount - FUSE Control File System was skipped because of an unmet condition check (ConditionPathExists=/sys/fs/fuse/connections).
systemd[1]: sys-kernel-config.mount - Kernel Configuration File System was skipped because of an unmet condition check (ConditionPathExists=/sys/kernel/config).
systemd[1]: systemd-repart.service - Repartition Root Disk was skipped because no trigger condition checks were met.
systemd[1]: Starting systemd-sysctl.service - Apply Kernel Variables...
EXT4-fs (sda2): re-mounted. Opts: (null)
```

```
systemd[1]: (This warning is only shown for the first unit using IP firewalling.)
systemd[1]: Starting systemd-journald.service - Journal Service...
systemd[1]: Starting systemd-modules-load.service - Load Kernel Modules...
systemd[1]: Starting systemd-remount-fs.service - Remount Root and Kernel File Systems...
systemd[1]: Starting systemd-udev-trigger.service - Coldplug All udev Devices...
systemd[1]: Mounted dev-mqueue.mount - POSIX Message Queue File System.
systemd[1]: Mounted sys-kernel-debug.mount - Kernel Debug File System.
systemd[1]: Finished fake-hwclock.service - Restore / save the current clock.
systemd[1]: modprobe@configfs.service: Deactivated successfully.
systemd[1]: Finished modprobe@configfs.service - Load Kernel Module configfs.
systemd[1]: modprobe@dm_mod.service: Deactivated successfully.
systemd[1]: Finished modprobe@dm_mod.service - Load Kernel Module dm_mod.
systemd[1]: modprobe@drm.service: Deactivated successfully.
systemd[1]: Finished modprobe@drm.service - Load Kernel Module drm.
systemd[1]: modprobe@efi_pstore.service: Deactivated successfully.
systemd[1]: Finished modprobe@efi_pstore.service - Load Kernel Module efi_pstore.
systemd[1]: modprobe@fuse.service: Deactivated successfully.
systemd[1]: Finished modprobe@fuse.service - Load Kernel Module fuse.
systemd[1]: modprobe@loop.service: Deactivated successfully.
systemd[1]: Finished modprobe@loop.service - Load Kernel Module loop.
systemd[1]: Finished systemd-modules-load.service - Load Kernel Modules.
systemd[1]: sys-fs-fuse-connections.mount - FUSE Control File System was skipped because of an unmet condition check (ConditionPathExists=/sys/fs/fuse/connections).
systemd[1]: sys-kernel-config.mount - Kernel Configuration File System was skipped because of an unmet condition check (ConditionPathExists=/sys/kernel/config).
systemd[1]: systemd-repart.service - Repartition Root Disk was skipped because no trigger condition checks were met.
systemd[1]: Starting systemd-sysctl.service - Apply Kernel Variables...
EXT4-fs (sda2): re-mounted. Opts: (null)
```

Raspbian GNU/Linux 12 raspberrypi tty1

My IP address is 10.0.2.15 ::ffff:127.0.1.1

raspberrypi login: pi

Password:

Linux raspberrypi 4.19.50+ #1 Tue Nov 26 01:49:16 CET 2019 armv6l

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/*copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.

SSH is enabled and the default password for the 'pi' user has not been changed.
This is a security risk - please login as the 'pi' user and type 'passwd' to set a new password.

pi@raspberrypi:~\$

Una vez iniciado, se conecta por SSH desde una segunda terminal:

```
ssh -p 5022 pi@127.0.0.1
```

Durante el desarrollo del practico se presentó el principal obstáculo técnico: la incompatibilidad de arquitectura entre las herramientas de compilación de la PC host y el núcleo corriendo en el entorno emulado.

Para materializar esta solución, nos situamos en el directorio raíz de nuestro entorno de trabajo, instalamos el conjunto de herramientas correcto y clonamos de manera superficial el repositorio oficial de Linux Raspberry Pi:

```
cd ~/noTengoGrupo-SdC/tp5_device_drivers

# Instalar el compilador cruzado específico para ARMv6
sudo apt install -y gcc-arm-linux-gnueabi

# Clonar la rama exacta del kernel compatible con nuestra emulación
git clone --depth=1 --branch rpi-4.19.y
https://github.com/raspberrypi/linux rpi-kernel-src
```

Para simplificar y estandarizar el proceso de construcción del controlador, nos situamos en la carpeta del driver (~/noTengoGrupo-SdC/tp5_device_drivers/driver) y creamos manualmente un archivo de configuración denominado Makefile.

Una vez conectados por SSH, el plan inicial del laboratorio requería la construcción de un controlador nativo en C denominado CDD.c. Para evitar escribir comandos de compilación extremadamente largos y propensos a errores en la terminal de la PC host, nos situamos en la carpeta del driver

(~/noTengoGrupo-SdC/tp5_device_drivers/driver) y creamos manualmente un archivo de texto llamado Makefile utilizando el editor de consola nano Makefile.

```
obj-m += CDD.o

# Esta línea deshabilita por completo el profiling/ftrace que genera el
__gnu_mcount_nc
ccflags-y := -fno-optimize-sibling-calls

all:
    make -C
/home/gustavo/noTengoGrupo-SdC/tp5_device_drivers/rpi-kernel-src ARCH=arm
CROSS_COMPILE=arm-linux-gnueabi- CONFIG_FUNCTION_TRACER=n M=$(PWD) modules

clean:
    make -C
/home/gustavo/noTengoGrupo-SdC/tp5_device_drivers/rpi-kernel-src ARCH=arm
CROSS_COMPILE=arm-linux-gnueabi- M=$(PWD) clean
```

Una vez guardado el Makefile en la carpeta driver, procedemos a ejecutar la automatización simplemente escribiendo en la terminal de nuestra PC host:

El driver implementa un Character Device que expone `/dev/CDD_GPIO`. Un kernel timer muestrea la señal cada 1 segundo. En QEMU, donde no hay GPIO físico, la señal se simula alternando entre 0 y 1.

Funciones implementadas:

cdd_init: Ejecutada al hacer ``insmod``. Registra el dispositivo, crea `/dev/CDD_GPIO` automáticamente e inicia el timer.

cdd_exit: Ejecutada al hacer ``rmmod``. Libera el timer, el nodo de dispositivo y el número mayor.

my_open: Llamada cuando un proceso abre `/dev/CDD_GPIO`. Registra el evento en el log del kernel.

my_close: Llamada cuando un proceso cierra el archivo.

my_read: Devuelve el último valor muestreado (0 o 1) al proceso de usuario.

my_write: Recibe ``"1"`` o ``"2"`` para seleccionar cuál señal sensar.

timer_callback: Ejecutada automáticamente cada 1 segundo. Actualiza el valor de la señal activa.

Los archivos compilados se transfieren a la Raspberry Pi virtual mediante scp:

```
scp -P 5022 driver/CDD.ko pi@127.0.0.1:~/  
scp -P 5022 app/signal_server.py pi@127.0.0.1:~/
```

```
gustavo@gustavo-Lenovo-V145-15AST:~/noTengoGrupo-SdC/tp5_device_drivers/rpi-kernel-src$ # 1. Volver a la carpeta del driver  
cd ~/noTengoGrupo-SdC/tp5_device_drivers/driver  
  
# 2. Compilar el módulo nativo en C usando las fuentes preparadas  
make  
make -C /home/gustavo/noTengoGrupo-SdC/tp5_device_drivers/rpi-kernel-src ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- CONFIG_FUNCTION_TRACER=n M=/home/gustavo/noTengoGrupo-SdC/tp5_device_drivers/driver modules  
make[1]: se entra en el directorio '/home/gustavo/noTengoGrupo-SdC/tp5_device_drivers/rpi-kernel-src'  
  
WARNING: Symbol version dump ./Module.symvers  
is missing; modules will have no dependencies and modversions.  
  
CC [M] /home/gustavo/noTengoGrupo-SdC/tp5_device_drivers/driver/CDD.o  
Building modules, stage 2.  
MODPOST 1 modules  
CC [M] /home/gustavo/noTengoGrupo-SdC/tp5_device_drivers/driver/CDD.mod.o  
LD [M] /home/gustavo/noTengoGrupo-SdC/tp5_device_drivers/driver/CDD.ko  
make[1]: se sale del directorio '/home/gustavo/noTengoGrupo-SdC/tp5_device_drivers/rpi-kernel-src'  
gustavo@gustavo-Lenovo-V145-15AST:~/noTengoGrupo-SdC/tp5_device_drivers/driver$ scp -P 5022 CDD.ko pi@127.0.0.1:~/  
pi@127.0.0.1's password:  
CDD.ko 100% 7732 5.2KB/s 00:01  
gustavo@gustavo-Lenovo-V145-15AST:~/noTengoGrupo-SdC/tp5_device_drivers/driver$ scp -P 5022 ../app/signal_server.py pi@127.0.0.1:~/  
pi@127.0.0.1's password:  
signal_server.py 100% 5979 156.8KB/s 00:00  
gustavo@gustavo-Lenovo-V145-15AST:~/noTengoGrupo-SdC/tp5_device_drivers/driver$
```

Dentro de la VM, se carga el driver y se verifica la carga correcta en el log del kernel:

```
sudo insmod CDD.ko  
dmesg | tail -10
```

Se dan permisos para que la aplicación de usuario pueda leer y escribir:

```
sudo chmod 666 /dev/CDD_GPIO
```

```
cat /dev/CDD_GPIO

sleep 2 && cat /dev/CDD_GPIO

echo "2" > /dev/CDD_GPIO
cat /dev/CDD_GPIO
```

La aplicación `signal_server.py` realiza las siguientes funciones:

- Lee `/dev/CDD_GPIO` cada segundo en un hilo separado
- Sirve una interfaz web en el puerto 8080
- Envía los datos al navegador usando Server-Sent Events (SSE)
- Permite seleccionar entre Señal 1 y Señal 2 desde el navegador
- Resetea el gráfico automáticamente al cambiar de señal

Se ejecuta dentro de la VM:

```
python3 ~/signal_server.py
```

```
pi@raspberrypi:~ $ python3 ~/signal_server.py
Iniciando lector de dispositivo...
Servidor web en http://0.0.0.0:8080
```



Conclusión:

En este trabajo práctico se exploró en detalle la creación y desarrollo de un Character Device Driver (CDD) para Linux. El recorrido progresivo desde los conceptos básicos hasta un driver funcional permitió comprender los mecanismos internos del kernel para la gestión de dispositivos.

El principal desafío técnico fue la incompatibilidad de arquitectura ARM durante la compilación cruzada. El kernel de QEMU corría en ARMv6 mientras que el compilador por defecto (arm-linux-gnueabi) genera código para ARMv7. La solución fue identificar el compilador correcto (arm-linux-gnueabi) y los headers exactos del kernel 4.19, lo que demuestra que en compilación cruzada la precisión de la cadena de herramientas es tan importante como el código fuente en sí.